

TP 2 – Automates

△ **Attention.** Pour ce TP, vous pouvez utiliser les fonctions des modules `List`, `Array` et `Hashtbl`. Utilisez `Zeal` pour avoir accès à la documentation de ces modules.

△ **Attention.** Il est important de lire le rappel sur la compilation OCaml pour comprendre le fonctionnement des modules qui sont définis pour ce TP (document disponible sur Cahier-De-Prépa).

I Manipulations de bases des automates

Les sources pour ce TP sont disponibles sur Cahier-De-Prépa. Elles définissent trois types d'automates, chacun dans son propre module, avec ses propres fonctions.

Automate est le module des automates (non-déterministes) vu au début du cours.

Le module définit un type `'a Automate.t` qui est polymorphe en le type des lettres. Dans cette implémentation, étant donné un état q et une lettre a , on peut obtenir la liste des états atteints par lecture de a depuis l'état q .

AutomateAsynchrone est le module des automates avec ε -transitions.

Le module définit un type `'a AutomateAsynchrone.t` qui est polymorphe en le type des lettres. Dans cette implémentation, étant donné un état q et une lettre a , on peut obtenir la liste des états atteints par lecture de a depuis l'état q . On peut aussi obtenir les transitions immédiates (ε -transitions) entre états.

AutomateDeterministe est le module des automates déterministes.

Le module définit un type `'a AutomateDeterministe.t` qui est polymorphe en le type des lettres. Dans cette implémentation, étant donné un état q et une lettre a , on peut obtenir l'unique état atteint par lecture de a depuis l'état q , s'il existe. Dans cette implémentation, l'état 0 désigne toujours l'état initial.

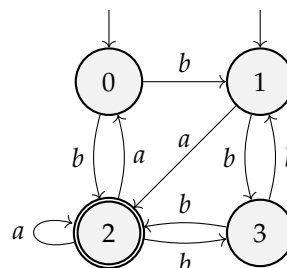
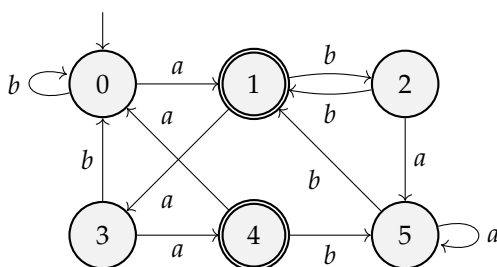
Exercice 1. Utilisation des modules en OCaml

On travaille sur le `tests.ml`.

- Q1.** Tester la compilation de `tests.ml` en donnant tous les modules en argument de la ligne de compilation.
[Optionnel] Tester l'auto-complétion de votre éditeur pour les modules, dans le fichier `tests.ml`.

Les modules définissent chacun un type pour leur automate, des fonctions pour ajouter des transitions entre états et une fonction pour afficher un automate dans le terminal.

Dans `tests.ml`, on fournit une variable représentant l'automate `a0`, représenté à gauche ci-dessous, ainsi qu'un automate `a1`. On considère également l'automate `a2` représenté à droite.



- Q2.** Dessiner l'automate avec ε -transition défini dans la variable `a1`.

- Q3.** En s'inspirant du code écrit pour définir l'automate `a0`, définir l'automate non-déterministe `a2` donné ci-dessus à droite. Il sera donc de type `char Automate.t`.

Exercice 2. Graphe et émondage

Les premières questions sont à écrire dans un nouveau fichier `graphe.ml`.

- Q4.** Définir un type `graphe` pour représenter des graphes en liste d'adjacence. Les sommets seront représentés par des entiers en partant de 0.
- Q5.** Définir une fonction `parcours : graphe -> int list -> bool array` qui effectue un parcours en profondeur depuis chaque sommet donné dans la liste en argument. La fonction renvoie un tableau de booléen indiquant pour chaque sommet du graphe s'il a été atteint lors du parcours ou non.
- Q6.** Définir une fonction `transpose : graphe -> graphe` qui crée le transposé d'un graphe.

Les questions suivantes sont à réaliser pour chaque type d'automate `t`, c'est-à-dire `AutomateDeterministe.t`, `Automate.t` et `AutomateAsynchrone.t`, dans le fichier `.ml` du module le définissant.

- Q7.** Écrire une fonction `vers_graphe : 'a t -> graphe` qui transforme un automate en un graphe orienté. Ce graphe a autant de sommets que l'automate avait d'états. Il "oublie" qui sont les états initiaux et acceptants, ainsi que les étiquettes sur les transitions. Pour chaque transition entre deux états il possède une arête entre les sommets correspondants.

Indication: les doublons dans les arêtes ne sont pas un problème, on pourra les laisser.

- Q8.** Compléter la fonction `accessibles : 'a t -> bool array` qui renvoie un tableau des états accessibles depuis les états initiaux.
- Q9.** De même compléter les fonctions `coaccessibles` et `utiles`.
- Q10.** Enfin, compléter la fonction `emonder : 'a t -> unit` qui modifie un automate pour supprimer toutes les transitions concernant des états qui ne sont pas utiles.
- Indication : Pour les automates non-déterministes, il peut être utile d'avoir une fonction auxiliaire `garder_utile : int list -> bool array -> int list` qui ne garde que les éléments de la liste dont la valeur associée dans le tableau vaut `true`.
- Q11. ★ [Bonus]** Faire un renommage des états restants pour ne pas garder les états déconnectés.

II Outils pour la détermination

Le but du reste de ce TP est de passer d'un automate mais potentiellement non-déterministe à un automate déterministe.

Exercice 3. Mise en place

On travaille sur le fichier `automates.ml`.

- Q12.** Écrire une fonction `est_deterministe : 'a Automate.t -> bool`, qui vérifie si un automate est déterministe ou non.
- Q13.** Tester la fonction sur plusieurs exemples dont `a2`.

Exercice 4. Parties de $\llbracket 0, N - 1 \rrbracket$

Pour lire un mot dans un automate non-déterministe, on a besoin de pouvoir considérer des ensembles d'états, puisque lire un même mot peut nous amener dans plusieurs états distincts.

Dans cette exercice, on écrit les fonctions permettant de travailler sur les parties de l'intervalle d'entiers $\llbracket 0, N - 1 \rrbracket$. Une partie P de cet intervalle est représentée par un tableau t de N booléens tel que $t[i]$ est vrai si et seulement si $i \in P$.

(Les fonctions peuvent être ajoutées directement dans le fichier `automates.ml` ou dans un nouveau fichier.)

- Q14.** Écrire une fonction `appartient : int -> bool array -> bool` telle que `appartient i p` teste si l'entier i est dans la partie représentée par le tableau booléen p .

- Q15. Écrire une fonction `vide : int -> bool array` telle que `vide n` construit la partie vide de l'intervalle $[0, n - 1]$
- Q16. Écrire une fonction `est_vide : bool array -> bool` testant si une partie représentée par un tableau de booléens est vide.
- Q17. Écrire une fonction `union : bool array -> bool array -> bool array` réalisant l'union de deux parties de $\llbracket 0, N - 1 \rrbracket$ dans un nouveau tableau.
- Q18. Similairement, écrire une fonction `intersection : bool array -> bool array -> bool array` réalisant l'intersection de deux parties de $\llbracket 0, N - 1 \rrbracket$ dans un nouveau tableau.
- Q19. Écrire une fonction `ajoute : bool array -> bool array -> unit` telle que `ajoute e1 e2` qui ajoute dans la partie représentée par `e1` les éléments de la partie représentée par `e2`. (A la fin, `e1` est l'union des deux parties)
- Q20. Écrire une fonction `construit : int list -> int -> bool array` telle que `construit l n` renvoie un tableau de taille `n` représentant l'ensemble des éléments contenus dans la liste `l`.

Exercice 5. Lecture d'un mot

On va maintenant utiliser les fonctions de l'exercice précédent. En effet, on sait que lire un mot dans un automate non-déterministe peut nous amener dans plusieurs états. Si on appelle Q l'ensemble des états d'un automate, on va considérer qu'on lit des mots en gardant trace des états où l'on peut être grâce à un tableau de booléens qui représente une partie de Q . Pour cet exercice, on se place dans `automate.ml`, le type `t` désigne donc le type des automates.

- Q21. Écrire une fonction `lire_lettre_etat : 'a t -> int -> 'a -> bool array` telle que l'appel `lire_lettre_etat a q b` renvoie la partie correspondant à l'ensemble des états où l'on peut se trouver après avoir lu `b` dans l'état `q` de l'automate `a`.
- Q22. Écrire une fonction `lire_mot : 'a t -> bool array -> 'a mot -> bool array` telle que l'appel `lire_mot a p u` effectue la lecture d'un mot depuis tous les états de la partie `p`.
- Q23. Écrire une fonction `accepte_mot : 'a t -> 'a mot -> bool` qui teste si un mot est accepté par un automate non-déterministe.

III Déterminisation

Pour ne pas répéter les opérations sur les parties de la section précédente à chaque test, on va tenter de déterminer nos automates.

Exercice 6. Déterminisation

On rappelle que si $\mathcal{A} = (Q, \Sigma, I, F, \delta)$, alors l'automate déterminisé a pour ensemble d'états l'ensemble des parties de Q . Chacun de ces états, c'est-à-dire chaque partie P de Q , peut être représenté par un tableau de booléens d'après ce qui précède. On peut lire ce tableau de booléen comme un nombre en binaire.

Pour cet exercice, on se place dans `automate.ml`, le type `t` désigne donc le type des automates.

- Q24. Écrire une fonction `vers_entier : bool array -> int` qui prend un tableau de booléens, l'analyse comme si c'était un nombre en binaire, et renvoie l'entier correspondant. Par exemple, le tableau `[|true;true;false;false|]` peut être interprété comme le nombre en binaire 1100, et la fonction précédente renvoie alors 12.

Remarque : On peut choisir le lire de droite à gauche ou de gauche à droite, ce choix n'aura pas d'influence sur l'algorithme final.

- Q25. Écrire une fonction `vers_partie : int -> int -> bool array` telle que `vers_partie p n` encode en binaire l'entier `p`, au moyen d'un tableau de booléens de taille `n`.

Remarque : Cette fonction doit bien être l'inverse de la précédente.

Q26. Écrire une fonction `puissance : int -> int -> int`, telle que `puissance p n` calcule p^n .

[Bonus] Pourquoi ne pas retravailler l'exponentiation rapide ?

Q27. ★ Écrire une fonction `determine : 'a t -> 'a AutomateDeterministe.t`, qui construit un automate déterministe à partir de l'automate fourni en argument.

Indication : Garder traces des états de l'automate déterminisé déjà traités à l'aide d'un tableau de booléens de la bonne taille.

IV Applications [Bonus]

Exercice 7. Machines de Moore★

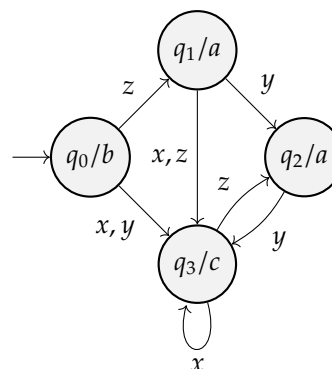
Informellement, une machine de Moore peut être vu comme un automate fini déterministe, sans états acceptants, mais où chaque état se voit attribuer une lettre d'un "alphabet de sortie Ω " (en plus des lettres de Σ qui étiquettent les transitions). Chaque fois qu'on arrive dans un état, on "produit" la lettre de sortie associée. Ainsi, en lisant un mot de Σ^* avec la machine, on va produire un mot de Ω^* . On peut donc voir une machine de Moore comme un outil réalisant une fonction partielle $f : \Sigma^* \rightarrow \Omega^*$. Formellement, une machine de Moore est un sextuplet $(Q, i, \Sigma, \Omega, \delta, \lambda)$ où :

- Q est l'ensemble d'états, et $i \in Q$.
- Σ est l'alphabet d'entrée (sur lequel on lit des mots, comme dans un automate) et Ω est l'alphabet de sortie (celui des mots produits par la machine).
- $\delta : Q \times \Sigma \rightarrow Q$ est la fonction de transition qui permet d'avancer en lisant des lettres de Σ
- $\lambda : Q \rightarrow \Omega$ est la fonction de sortie, qui à chaque état associe la lettre produite en arrivant dessus.

Pour un mot d'entrée $u \in \Sigma^*$ que la machine peut lire, on va suivre un unique chemin $q_0 \xrightarrow{u_1} q_1 \dots q_{n-1} \xrightarrow{u_n} q_n$, et le mot de sortie correspondant, noté $f(u)$, s'obtient par la concaténation $\lambda(q_1) \dots \lambda(q_n)$ (la sortie de l'état initial n'est pas prise en compte).

Voici un exemple de Machine de Moore, avec $\Sigma = \{x, y, z\}$ et $\Omega = \{a, b, c\}$. Pour chaque état q , on représente graphiquement l'état avec l'indication $q/\lambda(q)$.

Dans cet exemple, on aurait $f(zyyx) = aacc$; $f(yxz) = cca$, etc.



Q1. Dans un nouveau fichier `moore`, et en s'inspirant du type `'a t` du fichier `automates.ml`, définir un type `('a, 'b) moore` permettant de représenter une machine de Moore. Reprendre le type `type 'a mot = 'a list`.

Q2. Définir dans une variable `ex_moore` la machine de Moore donnée en exemple précédemment. On considèrera les lettres de Σ et de Ω comme des caractères.

Q3. Définir une fonction `lire_lettre : ('a, 'b) moore -> int -> 'a -> (int * 'b) option` tel que `lire_lettre m q x` effectue la lecture de la lettre `x` dans l'état `q` et renvoie la paire `Some (q', b)` représentant l'état d'arrivée et sa sortie, si la transition est possible, `None` sinon.

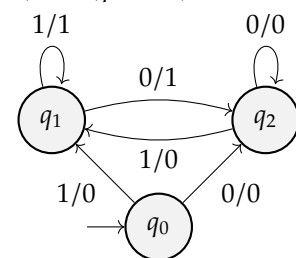
- Q4.** Définir une fonction `lire_mot : ('a,'b) moore -> 'a mot -> ('b mot) option`, qui effectue la lecture d'un mot de type 'a depuis l'unique état initial d'une machine de Moore, et renvoie le mot de type 'b correspondant à la sortie.
- Q5.** On pose $\Sigma = \Omega = \{a, b\}$. Dessiner une machine de Moore qui, pour chaque mot u , produit le mot obtenu en remplaçant chaque occurrence de a par un b et réciproquement. L'implémenter et la tester.
- Q6.** On pose $\Sigma = \{a, b\}$ et $\Omega = \{b, x, y\}$. Dessiner une machine de Moore qui, pour chaque mot u , produit le mot obtenu en remplaçant les occurrences d'ordre impair de a (le 1er a du mot, le 3ème a , le 5ème, etc...) par des x , et les occurrences d'ordre pair de a par des y . L'implémenter et la tester.

Exercice 8. Machines de Mealy★

Formellement, une machine de Mealy se définit exactement comme une Machine de Moore, mais on considère cette fois λ comme une fonction de $Q \times A$ dans B , avec la sémantique suivante : quand on part de q en lisant une lettre a , on produit la sortie $\lambda(q, a)$.

Pour un mot d'entrée $u \in \Sigma^*$ que la machine peut lire, on va suivre un unique chemin $q_0 \xrightarrow{u_1} q_1 \dots q_{n-1} \xrightarrow{u_n} q_n$, et le mot de sortie correspondant, noté $f(u)$, s'obtient par la concaténation $\lambda(q_0, u_1) \dots \lambda(q_{n-1}, u_n)$.

Voici un exemple de Machine de Mealy, avec $\Sigma = \Omega = \{0, 1\}$. Pour représenter les sorties, comme elles dépendent de l'état et de la lettre qu'on a lu pour y rentrer, on les représentera plutôt sur les transitions, sous la forme x/a , avec la sémantique "en prenant cette transition, on doit lire la lettre x , et on produit la sortie a ".



- Q1.** Dans un nouveau fichier `mealy.ml`, définir un type `(a,'b) moore` modélisant les machines de Mealy. Reprendre le type `type 'a mot = 'a list`.
- Q2.** Implémenter une fonction permettant de lire un mot dans une machine de Mealy, et produisant la sortie correspondante.
- Q3.** Que fait la machine de Mealy ci-dessus ? L'implémenter et la tester.
- Q4.** Dessiner, implémenter et tester une machine de Mealy qui réalise la fonction $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ suivante : pour tout $u = u_1 \dots u_n$, $f(u) = v_1 \dots v_n$ tel que pour tout i , v_{i+1} s'obtient par le "ou exclusif" de u_i et de u_{i+1} . (On aura toujours $v_1 = 0$ par convention)

V Berry-Sethi

Pour cette section, on travaillera dans le fichier `berrysethi.ml` disponible sur Cahier-de-Prépa.

Exercice 9. Linéarisation et langage local

Le fichier définit le type `expr` comme vu en cours en spécifiant que l'alphabet Σ sont les lettres du type `char`.

Le type `lin_expr` sert à représenter des expressions régulières linéarisées : chaque lettre est un couple correspondant à une lettre de notre alphabet Σ et d'un indice. Ce type de couple, appelé caractère indicé dans la suite, est défini par le type `indexed_char`.

- Q1.** Compléter la définition de la variable `epsilon` pour obtenir une expression régulière reconnaissant exactement ε .
- Q2.** Compléter la fonction `compte_char` qui compte le nombre de lettres dans l'expression régulière donnée. Par exemple $(aa|b)^2$ contient six lettres.
- Q3.** Compléter la fonction `linearise` qui linéarise une expression régulière. (On n'a pas besoin de la fonction précédente.)

Indication: La fonction doit déclarer un compteur et une fonction auxiliaire récursive qui utilisera ce compteur.

- Q4. Compléter la fonction `reconnait_epsilon` qui vérifie si le mot vide est accepté par l'expression régulière.
- Q5. Compléter les fonctions `premiers` et `derniers` qui renvoient les ensembles caractéristiques P et D de l'expression régulière linéaire sous forme de listes de caractères indicés.
- Q6. Compléter la fonction `facteurs` qui renvoie l'ensemble caractéristique F de l'expression régulière linéaire sous forme d'une liste de paires de caractères indicés.

Exercice 10. Vers l'automate

Le fichier définit un type `automate` pour représenter les automates de Glushkov de façon simplifié. On sait en particulier que l'état 0 est toujours l'unique état initial et, dans le cadre de ce TP où Σ est l'ensemble des `char`, on se permet d'utiliser la conversion entre `char` et `int` pour la relation de transition.

△ **Attention.** C'est bien les `char` qui sont convertis en `int` pour la relation de transition. À aucun moment les indices liés à la linéarisation ne seront utilisés dans les transitions.

- Q7. Lire attentivement la définition et le commentaire associé au type `automate`. S'agit-il d'une définition en matrice d'adjacence ou liste d'adjacence ? Comment obtenir le nombre d'états de l'automate ?
- Q8. Compléter la fonction `accepte` qui prend en argument un `automate` et une chaîne de caractère et vérifie si le premier accepte la seconde. On utilisera la fonction `Char.code` pour convertir les caractères de la chaîne en indice du tableau de transition. *Dans le cas d'une approche itérative, l'exception `No_transition` est disponible.*
- Q9. Compléter la fonction `construire` qui, à partir d'une expression régulière (non-linéarisée) construit un automate reconnaissant le même langage. On ne construira pas l'automate local intermédiaire mais directement l'automate de Glushkov.
- Q10. Tester les fonctions sur plusieurs exemples, par exemple $(aa^*|(b|cb)^*)$.